

DYN-EYE

System Architecture Document

1. System Overview

DYN-EYE is an unknown defect discovery and iterative model improvement system for industrial visual inspection. It takes a set of input images, uses an existing YOLOv8 object detection model to separate images into known and unknown categories, then uses a Vision Language Model (VLM) to annotate the unknown images with bounding boxes around potential defects. Those bounding boxes are cropped out, embedded into a vector space using DINOv2, filtered for novelty against a FAISS index of known defects, and then clustered using HDBSCAN. The resulting clusters are presented in a web dashboard where a human operator names them. Once named, a retraining agent exports the labels into YOLO format, gets training hyperparameter recommendations from an LLM, fine-tunes the YOLO model, and deploys the updated model.

1.1 Technology Stack

Component	Technology
Object Detection	YOLOv8 (Ultralytics)
Vision Language Model	Gemma 4-31b-it (Google GenAI API)
Dynamic Prompt Generation	Llama-3.3-70b-versatile (Groq API)
Feature Embeddings	DINOv2 ViT-S/14 (Facebook, 384-dim)
Nearest-Neighbor Index	FAISS IndexFlatL2 (Facebook)
Clustering	HDBSCAN + UMAP/PCA + KMeans fallback
Pipeline Orchestration	LangGraph (StateGraph)
LLM Training Advisor	Groq Llama-3.3-70b / Gemini 2.0 Flash
Experiment Tracking	MLflow (local file store)
Dataset Versioning	DVC (Data Version Control)
Dashboard	FastAPI + HTML/CSS/JS
Model Registry	Custom JSON-based local registry

1.2 Directory Structure

```
dyn-eye/  
■■■ config.py # All paths, thresholds, API keys  
■■■ models/best.pt # Active YOLO model weights
```

- models/versions/ # Versioned model checkpoints
- data/
 - ■■■ input_images/ # Raw inspection images
 - ■■■ crops/ # VLM-extracted defect crops
 - ■■■ clusters/ # HDBSCAN cluster folders + manifest
 - ■■■ faiss_index/ # FAISS index + label files
 - ■■■ yolo_dataset/ # Exported YOLO train/val split
 - ■■■ known_defect_crops/ # Reference crops for FAISS setup
 - ■■■ known_defects.json # Known defect class registry
- src/pipeline/nodes/ # 8 LangGraph discovery nodes
- src/pipeline/graph.py # Discovery pipeline graph definition
- src/retraining/agent.py # Retraining agent graph definition
- src/retraining/llm_advisor.py # LLM hyperparameter advisor
- src/features/ # DINOv2 extractor, FAISS manager
- dashboard/ # FastAPI dashboard app

2. Discovery Pipeline Diagram

The discovery pipeline is a LangGraph StateGraph with 8 nodes executed in a linear sequence. Each node reads from and writes to a shared state dictionary. The pipeline is defined in `src/pipeline/graph.py` and invoked via `run_discovery_pipeline()`.

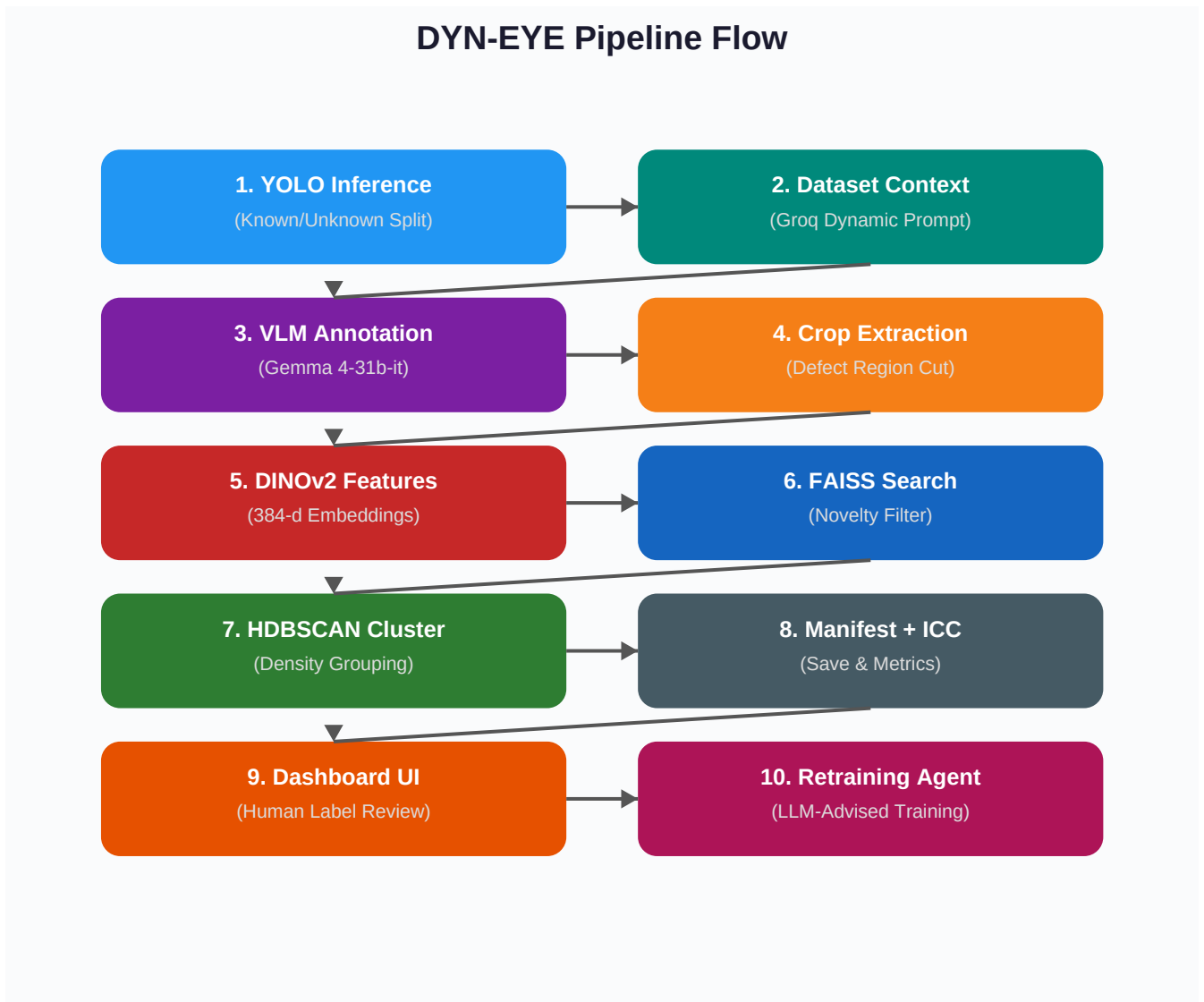


Figure 1: Discovery pipeline flow. Nodes 1–8 run automatically. Node 9 (Dashboard) requires human interaction to name clusters. Node 10 (Retraining) is triggered from the dashboard after labelling.

2.1 Retraining Agent Diagram

The retraining agent is a separate LangGraph StateGraph with 7 nodes. It contains conditional edges: if dataset validation fails, the pipeline stops. If training fails, deployment is skipped.

Retraining Agent Flow

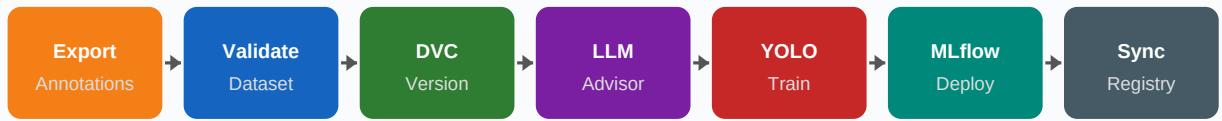


Figure 2: Retraining agent flow. Conditional gates after Validate and Train nodes can halt the pipeline early if validation fails or training produces no usable model.

3. Discovery Pipeline — Detailed Node Descriptions

Each node below is described exactly as implemented in the source code. The Input/Output table shows the state keys that each node reads and writes.

Node 1: YOLO Inference — `src/pipeline/nodes/yolo_inference.py`

This node runs the current YOLOv8 model (`models/best.pt`) over every image in the input directory. It classifies each image as either **known** (the model detected a defect class it was trained on, with confidence above the threshold) or **unknown** (no confident known-class detection).

The model is loaded using the Ultralytics YOLO API. Images are processed in batches of 16. For each image, every detection box is checked: if the predicted class name (lowercased) matches any entry in the known defects registry and the confidence score meets the threshold (default 0.30), the image is marked as known.

There is a **fallback stage**: if YOLO does not confidently detect a known class, the node checks if any crop file in `data/known_defect_crops/` has a filename prefix matching the current image stem. This bootstraps filtering for known defects before the model is fully fine-tuned on new classes.

The known defect names list is read dynamically from the known defects registry (`data/known_defects.json`) at the start of each run, so newly deployed classes are picked up automatically.

All unknown image paths are saved to `data/unknown_defects.json` for downstream use.

Direction	State Keys
Reads	<code>input_images_dir</code> , <code>use_cache</code>
Writes	<code>all_image_paths</code> , <code>known_image_paths</code> , <code>unknown_image_paths</code> , <code>yolo_raw_results</code> , <code>known_defects</code>

Node 2: Dataset Context (Dynamic VLM Prompt) —
src/pipeline/nodes/dataset_context.py

This node runs after YOLO inference and before VLM annotation. Its purpose is to generate a **domain-specific detection prompt** for the VLM, so that the VLM annotation step uses vocabulary and sensitivity appropriate to the current inspection domain rather than a generic static prompt.

It first builds a **pre-annotation context** dictionary containing: the inspection domain (from config, e.g. 'steel_casting'), the list of known defect class names, the count of unknown images found by YOLO, the novelty ratio (unknown count / total count), and **closed-loop VARS feedback metrics** from the previous pipeline run (CDS, BQS, DRS). A flag `high_novelty` is set to True if the novelty ratio exceeds 0.5.

If previous VARS sub-scores indicate quality bottlenecks (e.g. low BQS for overzoomed boxes, low CDS for background texture hallucination, or low DRS for imbalanced detection rates), specific **vars_corrective_directives** are dynamically injected into the context.

This context is sent to the **Groq API** (Llama-3.3-70b-versatile model) along with a meta-prompt that instructs the LLM to write a Gemini-compatible system prompt. The meta-prompt tells the LLM to include the inspection domain, list all known classes so the VLM can distinguish new defects from near-misses, apply any VARS corrective directives to optimize prompt quality, and if `high_novelty` is true, instruct the VLM to coin new descriptive labels.

The generated prompt is cached to disk (keyed by a SHA-256 hash of the context) so repeated runs with identical contexts do not re-call Groq. If Groq is unavailable (no API key, network error, or the groq package is not installed), the node silently falls back to the static prompt defined in `vlm_annotation.py`.

Direction	State Keys
Reads	known_defect_names, unknown_image_paths, all_image_paths
Writes	dataset_context, vlm_system_prompt (optional)

Node 3: VLM Annotation — `src/pipeline/nodes/vlm_annotation.py`

This node sends each unknown image to the **Gemma 4-31b-it** vision language model through the Google GenAI API for bounding-box defect detection. Images are processed **one at a time** (sequentially, not batched) with a configurable sleep interval between calls (default 4.5 seconds) to respect API rate limits.

The VLM is called with `response_mime_type='application/json'` and a Pydantic response schema (`InspectionReport`) that enforces the output format. The expected JSON has: `anomalies_found` (bool) and `findings` (list of objects, each with `box_2d` as `[ymin, xmin, ymax, xmax]` in 0–1000 scale, and `physical_traits` as a text description).

If the dynamic prompt was generated by the Dataset Context node, it is used. Otherwise the node falls back to a hardcoded static system prompt that instructs the VLM to detect structural/geometric violations, surface/texture violations, and tonal/material violations, with tight bounding boxes (5–8% margin).

Each image call has retry logic with exponential backoff (up to 5 retries). On 503 errors or network failures, the backoff is extended. If all retries fail for an image, it is recorded with an error flag and empty findings.

Post-Processing Box Merging: Raw bounding boxes undergo an automated overlap consolidation filter. If two predicted boxes overlap with an Intersection-over-Union (IoU) ≥ 0.30 or Intersection-over-Smaller-Area (IoS) ≥ 0.50 (nested boxes), they are merged into a unified bounding envelope `[min_y, min_x, max_y, max_x]` with combined physical traits. This eliminates redundant overlapping crops prior to feature extraction.

The VLM temperature is set to 0.1 for maximum determinism. All annotations are automatically cached to `data/vlm_cache.json` after processing completes, so a subsequent run with `use_cache=True` can skip the VLM entirely.

Direction	State Keys
Reads	<code>unknown_image_paths</code> , <code>use_cache</code> , <code>vlm_system_prompt</code>
Writes	<code>vlm_annotations</code> (list of dicts with <code>image_path</code> , <code>findings</code> , etc.)

Node 4: Crop Extraction — `src/pipeline/nodes/crop_extraction.py`

This node takes the bounding box annotations from the VLM and physically crops those regions out of the original images using OpenCV. Each crop is saved as a JPEG file in `data/crops/` with a naming convention of `{image_stem}_crop_{index:04d}.jpg`.

The VLM returns box coordinates in a 0–1000 scale relative to image dimensions. This node converts them to pixel coordinates by multiplying by the actual image height and width, then dividing by 1000. Coordinates are clamped to image boundaries.

Three quality filters are applied to each crop before saving:

- 1. Minimum size guard:** Crops smaller than 30×30 pixels (or area < 900 px²) are discarded as too small to be meaningful.
- 2. Overzoom guard:** If a crop covers more than 85% of the original image in both dimensions, it is discarded. This catches a common VLM failure mode where the model draws a box around the entire image.
- 3. Blur guard:** The Laplacian variance of the grayscale crop is computed. If it falls below 25.0, the crop is considered too blurry or uninformative and is discarded.

Metadata for each saved crop (source image path, bounding box in both raw and pixel coordinates, physical traits description, crop dimensions) is collected and passed downstream for manifest generation.

Direction	State Keys
Reads	<code>vlm_annotations</code> , <code>use_cache</code>
Writes	<code>crop_paths</code> (list of file paths), <code>crop_metadata</code> (list of dicts)

Node 5: DINOv2 Feature Extraction — `src/pipeline/nodes/feature_extraction.py`

This node encodes each crop image into a **384-dimensional feature vector** using the DINOv2 ViT-S/14 model (loaded from PyTorch Hub, Facebook Research). DINOv2 is a self-supervised vision transformer that produces dense, semantically meaningful embeddings without requiring task-specific fine-tuning.

Each image is preprocessed by resizing to 224×224, converting to a tensor, and normalizing with ImageNet statistics (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]). Images are processed in batches (default batch size 32) for efficiency.

The output vectors are **L2-normalized** using `torch.nn.functional.normalize(features, p=2, dim=-1)` so that cosine similarity between any two vectors equals their dot product. This normalization is critical for the FAISS distance computation and HDBSCAN clustering that follow.

The model runs on GPU if CUDA is available, otherwise CPU. All inference is done under `@torch.no_grad()` to avoid gradient computation.

The final output is a NumPy array of shape (N, 384) where N is the number of crops.

Direction	State Keys
Reads	<code>crop_paths</code>
Writes	<code>feature_vectors</code> (N×384 numpy array), <code>feature_crop_paths</code>

Node 6: FAISS Novelty Search — `src/pipeline/nodes/faiss_search.py`

This node queries every crop's 384-d embedding against a pre-built **FAISS IndexFlatL2** index of known defect embeddings. The index contains embeddings of reference crops stored in `data/known_defect_crops/`, organized by class subdirectory (e.g. `scratch/`, `dent/`).

For each query vector, FAISS returns the L2 distance to its single nearest neighbor (k=1 search). If this distance exceeds the novelty threshold (default 0.35 in L2 space of L2-normalized vectors), the crop is flagged as **novel** — meaning it does not closely resemble any known defect type.

The threshold of 0.35 on L2-normalized vectors corresponds roughly to a cosine similarity of 0.825. This was tuned to balance between catching truly new defect types and not flooding the system with minor variations of known defects.

If no FAISS index file exists on disk (first run before FAISS setup), all crops are treated as novel. This is a safe default that ensures the system works out of the box even without a pre-built reference database.

The FAISS index is rebuilt automatically after every successful model deployment or rollback, ensuring it stays synchronized with the deployed model's class set.

Direction	State Keys
Reads	feature_vectors, feature_crop_paths
Writes	faiss_distances, faiss_is_novel, novel_indices, total_detected_crops

Node 7: HDBSCAN Clustering — `src/pipeline/nodes/hdbscan_cluster.py`

This node groups the novel crop embeddings into clusters using density-based clustering. It is the most complex node in the pipeline with several stages:

Stage 1 — L2 Normalization: Novel feature vectors are L2-normalized (vectors with zero norm are left as-is to avoid division by zero).

Stage 2 — Dimensionality Reduction: If there are ≥ 30 crops, UMAP is used to project the 384-d vectors into a 10-dimensional space (cosine metric, fixed `random_state=42`, `n_jobs=1` for determinism). For fewer than 30 crops, PCA is used as a deterministic fallback. For ≤ 10 crops, no reduction is applied.

Stage 3 — Adaptive Parameter Tuning: A grid search over (`min_cluster_size`, `min_samples`) is performed. For each parameter combination, HDBSCAN is run and the resulting clustering is scored by silhouette coefficient (computed on L2-normalized high-dimensional features, not the reduced space). The combination with the best silhouette score wins, with ties broken in favor of more clusters. Candidate values are scaled to the dataset size.

Stage 4 — Clustering Execution: HDBSCAN is run with the tuned parameters (EOM cluster selection, euclidean metric). If the noise ratio exceeds 85% or no clusters are found, AgglomerativeClustering is used as a fallback (`distance_threshold=0.65`, average linkage). If only a single cluster results and there are ≥ 6 crops, a silhouette-optimized KMeans sweep over `k=2..8` is run.

Stage 5 — Noise Reassignment: Points labeled as noise (`label=-1`) are reassigned to the nearest cluster centroid if their cosine distance is below the reassignment threshold (default 0.50). Remaining unassigned points are placed in an 'unassigned' folder.

Stage 6 — Cluster Fingerprint Registry: Each cluster's L2-normalized centroid is compared against a persistent fingerprint registry (`data/clusters/cluster_registry.json`). If a centroid matches a stored fingerprint within the match threshold (cosine distance < 0.30), the match count is incremented. Otherwise, a new fingerprint is registered. This gives clusters persistent identity across pipeline runs.

Crop files are physically copied into cluster subdirectories (`data/clusters/cluster_000/`, etc.) for the dashboard.

Direction	State Keys
Reads	feature_vectors, feature_crop_paths, novel_indices
Writes	cluster_labels, cluster_folders, num_clusters, cluster_registry, cluster_tuned_params, unassigned_crops

Node 8: Manifest Save + ICC Metrics — src/pipeline/nodes/manifest_save.py

This is the final node in the discovery pipeline. It produces the `cluster_manifest.json` file that the dashboard reads, and computes cluster quality metrics.

Cohesion Score: For each cluster, the mean cosine similarity between every member vector and the cluster centroid is computed. This gives a 0–1 score where higher means the cluster is more visually consistent.

Global ICC (Intra-Class Correlation): Computed via a one-way ANOVA decomposition on the DINOv2 embeddings. The between-group sum of squares (`SS_between`) and within-group sum of squares (`SS_within`) are computed, then $ICC = (MS_between - MS_within) / (MS_between + (k - 1) \cdot MS_within)$ where `k` adjusts for unequal group sizes. This measures how well the clusters separate distinct defect types.

Global Silhouette Score: Computed on L2-normalized features with cosine metric (sklearn), excluding noise points. Requires at least 4 non-noise points across at least 2 clusters.

The manifest JSON maps each cluster folder name to its crops, including: crop file path, source image path, bounding box coordinates (both raw 0–1000 and pixel), physical traits text, and crop dimensions. It preserves any previously assigned defect names from the old manifest.

A separate `crop_to_source.json` mapping is saved, which records the normalized YOLO-format bounding box (center x, center y, width, height) for each crop relative to its source image. This is used during the export step of the retraining agent.

Direction	State Keys
Reads	cluster_folders, crop_metadata, run_id, vlm_system_prompt, feature_vectors, cluster_labels, n
Writes	vlm_cluster_results

4. Dashboard (Human-in-the-Loop)

The dashboard is a FastAPI web application that provides a visual interface for the human operator. It serves the following functions:

- **Cluster Review:** Displays each cluster as a grid of crop thumbnails. The operator can visually inspect whether the crops in a cluster actually represent the same defect type.
- **Cluster Naming:** The operator assigns a defect class name to each cluster (e.g. 'edge_crack', 'surface_pit'). This name is written into the `cluster_manifest.json`.
- **Crop Reassignment:** Individual crops can be moved between clusters or removed entirely if they are incorrect detections.
- **Pipeline Controls:** Start/stop the discovery pipeline (fresh run or cache mode), trigger the retraining agent, view live log streaming via Server-Sent Events (SSE).
- **Model Registry:** View all registered model versions with their metrics (mAP50, precision, recall), deployment status, and deployment history. Deploy or rollback to any version with a single click. Deployment automatically rebuilds the FAISS index and syncs the known defects registry.
- **Run Metrics:** Displays per-cluster cohesion scores, global ICC, global silhouette score, and per-node execution timing.

5. Retraining Agent — Detailed Node Descriptions

The retraining agent is a LangGraph StateGraph defined in `src/retraining/agent.py`. It is triggered from the dashboard after the human has named at least one cluster. It has 7 nodes with conditional edges.

Node R1: Export Annotations — `src/retraining/agent.py` → `export_node`

Reads the `cluster_manifest.json` and identifies all clusters that have been given a defect name by the human operator. For each named cluster, it maps the crops back to their original source images using the `crop_to_source.json` mapping.

The mapping produces YOLO-format label files: each source image gets a `.txt` file with lines of `class_id center_x center_y width height` (normalized 0–1 coordinates). Source images are copied into `data/yolo_dataset/images/{train|val}/` and label files into `data/yolo_dataset/labels/{train|val}/` with an 80/20 train/val split.

A `data.yaml` file is generated listing the class names and paths. If no named clusters are found, the node returns an error and the pipeline terminates.

Direction	State Keys
Reads	(manifest file on disk)
Writes	export_result

Node R2: Validate Dataset — *src/retraining/tools/dataset_validator.py*

Validates the exported YOLO dataset for structural correctness: checks that `data.yaml` exists and is parseable, that at least one class is listed, that image files referenced in the dataset exist, and that label files contain valid YOLO-format annotations (5 space-separated values per line, class IDs within range, coordinates in 0–1).

An additional safeguard checks that the training split contains at least 1 image. If the dataset has zero training images (e.g. due to stale absolute paths from a previous workspace), validation fails immediately with a descriptive error.

If validation fails, the conditional edge routes the pipeline to END, skipping all subsequent nodes.

Direction	State Keys
Reads	export_result
Writes	validation_result

Node R3: DVC Dataset Versioning — *src/retraining/tools/dvc_version.py*

Versions the `data/yolo_dataset/` directory using DVC (Data Version Control). It runs four shell commands: `dvc add` (track the dataset), `git add` (stage the `.dvc` file and `.gitignore`), `git commit` (commit with a timestamped message), and `git tag` (create an annotated tag like `dataset-v20260701_120000`).

If DVC is not initialized in the project, it runs `dvc init` first. Version metadata (tag, timestamp, paths) is saved to a JSON file under `logs/dataset_versions/`.

This step is skipped if dataset validation failed in the previous node.

Direction	State Keys
Reads	validation_result
Writes	dvc_result

Node R4: LLM Training Advisor — *src/retraining/llm_advisor.py*

This node queries an LLM to analyze the dataset metadata and recommend training hyperparameters. It first collects metadata by scanning the named clusters: number of classes, crops per class, total crops, min/max/avg crops per class, whether the base model exists, existing known defect names, and the total number of YOLO backbone layers (detected by loading the model checkpoint).

A detailed prompt is constructed that explains YOLO fine-tuning best practices and asks the LLM to output a JSON object with: `should_train` (bool), `reason` (str), and a `config` object containing: `epochs`, `batch_size`, `image_size`, `learning_rate` (`lr0`, `lrf`), `momentum`, `weight_decay`, `warmup_epochs`, `patience`, `optimizer_type`, `cosine_LR_flag`, `backbone_freeze_depth`, and 11 augmentation parameters (`mosaic`, `mixup`, `degrees`, `translate`, `scale`, `flipud`, `fliplr`, `hsv_h`, `hsv_s`, `hsv_v`, `augment_toggle`).

The prompt includes a heuristic suggestion for the freeze depth: `freeze = max(0, total_layers - max(1, total_crops // 15))`, and scaling guidelines (e.g. `<30 crops` → head-only training, `>200 crops` → allow most layers to adapt).

The node tries **Groq first** (Llama-3.3-70b-versatile, JSON mode), then **Gemini** (gemini-2.0-flash, JSON response MIME type), then a **heuristic fallback** that scales epochs, batch size, learning rate, freeze depth, and augmentation intensity based on dataset size thresholds (`< 50`, `50–200`, `200–500`, `> 500` crops).

The LLM response is validated and merged with defaults. If the LLM returns the generic default freeze value of 10, a smart override is applied using the heuristic formula. If the LLM advises against training (e.g. too few samples), the pipeline skips training.

Direction	State Keys
Reads	(manifest + dataset files on disk)
Writes	llm_recommendation (dict with <code>should_train</code> , <code>reason</code> , <code>config</code>)

Node R5: YOLO Fine-Tuning — *src/retraining/tools/train_yolo.py*

This node fine-tunes the YOLOv8 model using the Ultralytics training API. It merges hyperparameters from three sources with this priority: **user-specified overrides** (from the dashboard) > **LLM-recommended config** > **config.py defaults**.

Before training starts, Python garbage collection is triggered and the PyTorch CUDA cache is cleared to free memory. The training is configured with `workers=0` (main-thread data loading to avoid Windows multiprocessing memory duplication) and `cache=False` (no image caching to RAM).

Custom callbacks are registered on the YOLO model to stream training progress to the dashboard via the `LogStream`: `on_train_start`, `on_train_epoch_start`, `on_fit_epoch_end` (logs loss values), and `on_train_end`.

After training, the best model weights (`best.pt`) are copied to both the model versions directory and the active model path. Metrics (`mAP50`, `mAP50-95`, `precision`, `recall`) are extracted from the Ultralytics results object.

The trained model is also registered in the local Model Registry with full metadata (metrics, training config, source, class names, dataset stats).

Direction	State Keys
Reads	llm_recommendation, epochs, imgsz, batch_size, freeze
Writes	training_result (dict with <code>success</code> , <code>model_path</code> , <code>metrics</code> , <code>training_config</code>)

Node R6: MLflow Deployment — `src/retraining/tools/mlflow_deploy.py`

Registers the fine-tuned model in **MLflow** (local file-backed tracking store). It creates a new MLflow run, logs the training metrics, logs the model file as an artifact, creates a registered model version, and transitions it to the 'Production' stage (archiving any previous production versions).

The model is also copied to the active model path (`models/best.pt`) and to the versions directory. Deployment metadata (model name, version, run ID, timestamp, metrics) is saved to `logs/deployments/`.

The Model Registry's deployment status and deployment history are updated. If MLflow fails (e.g. missing dependency), the model is still copied locally as a fallback.

This node is skipped (via conditional edge) if training failed.

Direction	State Keys
Reads	training_result
Writes	deploy_result

Node R7: Sync Registry + Rebuild FAISS — `src/retraining/agent.py` → `sync_registry_node`

This is the final node of the retraining pipeline. It performs three tasks:

- 1. Register classes from the YOLO model:** Loads the newly deployed `best.pt` model and reads its class names dictionary. Any class names not already in the known defects registry are added.
- 2. Register classes from data.yaml:** Reads the `data/yolo_dataset/data.yaml` file and merges its class names into the registry. This catches any classes that might be in the training data but not in the model (edge case).
- 3. Rebuild FAISS index:** Re-extracts DINOv2 embeddings from all crops in `data/known_defect_crops/` and rebuilds the FAISS IndexFlatL2 index. This ensures the next discovery pipeline run will correctly identify crops that match the newly learned classes as 'known' rather than 'novel'.

The known defects registry (`data/known_defects.json`) is a persistent JSON file that tracks: the sorted list of defect class names, a version counter, a last-updated timestamp, and a history array of all additions with their sources. It is thread-safe (uses a threading Lock for writes).

Direction	State Keys
Reads	deploy_result, training_result
Writes	sync_result

6. Key Design Decisions

6.1 Why DINOv2 for Embeddings?

DINOv2 ViT-S/14 was chosen because it produces high-quality visual embeddings without any task-specific fine-tuning. Since the system is designed to discover *unknown* defect types (which by definition have no training labels), a self-supervised model that learns general visual features is more appropriate than a supervised model trained on a fixed set of classes. The 384-dimensional output is compact enough for efficient FAISS indexing while still being discriminative.

6.2 Why HDBSCAN over KMeans?

HDBSCAN does not require specifying the number of clusters in advance. Since the system discovers unknown defect types, the number of distinct defect classes in a batch of images is not known ahead of time. HDBSCAN also naturally identifies noise points (outliers that do not belong to any cluster), which is useful for filtering out VLM false positives. However, the system includes KMeans and Agglomerative Clustering as fallbacks for edge cases (single cluster, very small datasets, high noise ratio).

6.3 Why LLM-Advised Hyperparameters?

The training hyperparameters (learning rate, freeze depth, augmentation intensity, epoch count) should adapt to the dataset size and composition. A fixed configuration would either overfit small datasets or undertrain large ones. By providing the dataset metadata to an LLM and asking it to reason about the optimal configuration, the system can adapt without hardcoding rules for every possible scenario. The heuristic fallback ensures the system works even without LLM access.

6.4 Why Two Separate LangGraph Pipelines?

The discovery pipeline and the retraining agent are separate graphs because they have different trigger conditions: discovery runs over new input images, while retraining runs only after a human has labeled clusters. Separating them also means a retraining failure does not block future discovery runs, and the human labeling step naturally forms the boundary between the two workflows.

6.5 Known Defects Registry as Single Source of Truth

Rather than hardcoding defect class names in config, the system maintains a persistent JSON registry that grows over time. Every YOLO inference node reads from this registry at run time, so a newly deployed model's classes are immediately recognized as 'known' in the next discovery run. The registry is updated from three sources: the deployed YOLO model's class dictionary, the training data.yaml, and manual additions via the dashboard.

7. Configuration Reference

Key configuration values defined in `config.py`:

Parameter	Default Value	Purpose
YOLO_CONFIDENCE_THRESHOLD	0.30	Min confidence for YOLO known-class detection
VLM_MODEL_ID	gemma-4-31b-it	Vision Language Model for annotation
VLM_TEMPERATURE	0.1	VLM generation temperature (low = deterministic)
VLM_SLEEP_BETWEEN	4.5s	Delay between VLM API calls (rate limiting)
VLM_MAX_RETRIES	5	Max retry attempts per VLM call
FEATURE_DIM	384	DINOv2 ViT-S/14 embedding dimension
FAISS_NOVELTY_THRESHOLD	0.35	L2 distance above which = novel
HDBSCAN_MIN_CLUSTER_SIZE	3	Minimum cluster size for HDBSCAN
HDBSCAN_METRIC	euclidean	Distance metric for HDBSCAN
CLUSTER_MATCH_THRESHOLD	0.30	Cosine dist for fingerprint matching
CLUSTER_NOISE_REASSIGN	0.50	Cosine dist for noise reassignment
UMAP_N_COMPONENTS	10	Target dimensions after UMAP
UMAP_BATCH_THRESHOLD	30	Min crops to activate UMAP over PCA
GROQ_ADVISOR_MODEL	llama-3.3-70b	LLM for dynamic prompt + advisor
LLM_ADVISOR_MODEL	gemini-2.0-flash	Fallback LLM for training advice
LLM_MIN_CROPS_PER_CLASS	10	Min crops before LLM considers training